

Презентация по лабораторной работе №7

Основы информационной безопасности

Кузьмин Егор Витальевич, НКАбд-01-23

16 мая 2025

Российский университет дружбы народов, Москва, Россия

Информация

- Кузьмин Егор Витальевич
- студент группы НКАбд-01-23
- Российский университет дружбы народов

Освоить на практике применение режима однократного гаммирования

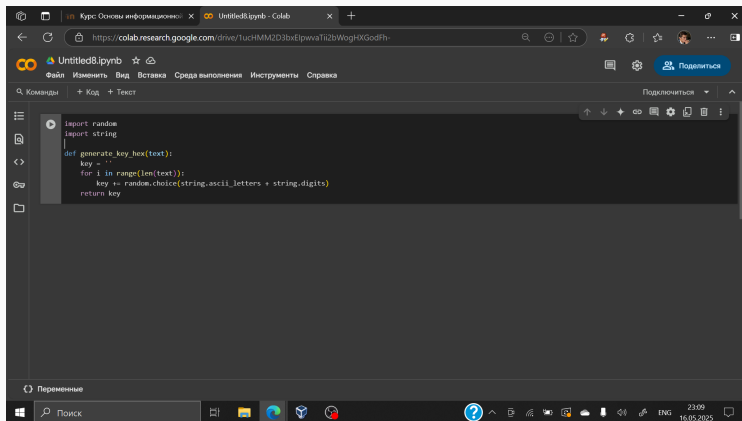
Нужно подобрать ключ, чтобы получить сообщение «С Новым Годом, друзья!». Требуется разработать приложение, позволяющее шифровать и дешифровать данные в режиме однократного гаммирования. Приложение должно:

1. Определить вид шифротекста при известном ключе и известном открытом тексте.
2. Определить ключ, с помощью которого шифротекст может быть преобразован в некоторый фрагмент текста, представляющий собой один из возможных вариантов прочтения открытого текста

Я выполнил лабораторную работу на языке программирования Python в гугл-коллаб, листинг программы и результаты выполнения приведены в отчете.

Выполнение лабораторной работы

Требуется разработать программу, позволяющее шифровать и дешифровать данные в режиме однократного гаммирования. Начнем с создания функции для генерации случайного ключа



The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL <https://colab.research.google.com/drive/TuchMM2D3bxElpwvaTii2bWogHXGodFh->. The notebook title is "Untitled8.ipynb". The menu bar includes "Файл", "Изменить", "Вид", "Вставка", "Среда выполнения", "Инструменты", and "Справка". The toolbar shows "Команды", "+ Код", "+ Текст", and a "Подключиться" button. The code editor contains the following Python code:

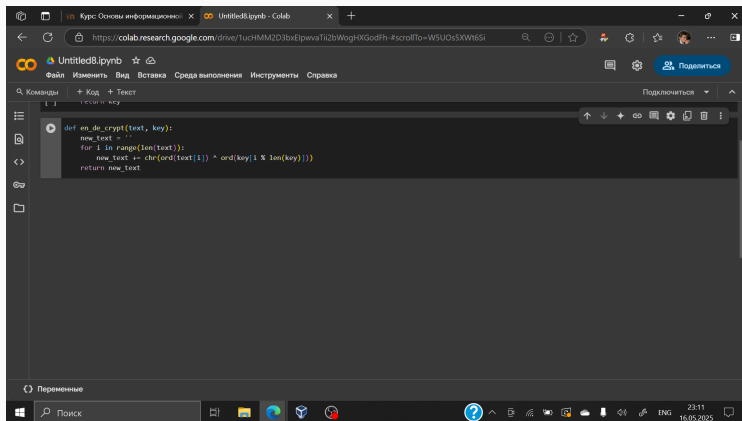
```
import random
import string

def generate_key_hex(text):
    key = ''
    for i in range(len(text)):
        key += random.choice(string.ascii_letters + string.digits)
    return key
```

At the bottom of the notebook, there is a "Переменные" (Variables) section which is currently empty. The Windows taskbar at the very bottom shows the search bar, task view, and several application icons, along with the system clock indicating 23:09 on 16.05.2025.

Выполнение лабораторной работы

Необходимо определить вид шифротекста при известном ключе и известном открытом тексте. Так как операция исключающего или отменяет сама себя, делаю одну функцию и для шифрования и для дешифрования текста



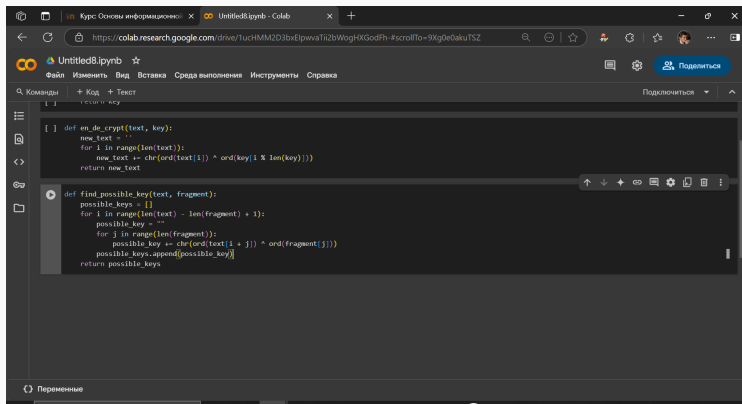
The screenshot shows a Google Colab notebook interface. The browser address bar displays the URL: <https://colab.research.google.com/drive/TuchMM2D3bxElpwvaTii2bWogHXGodFh-#scrollTo=W5UOs5XWt6Si>. The notebook title is "Untitled8.ipynb". The code editor contains the following Python function:

```
def en_de_crypt(text, key):  
    new_text = ''  
    for i in range(len(text)):  
        new_text += chr(ord(text[i]) ^ ord(key[i % len(key)]))  
    return new_text
```

The interface includes a menu bar with options like "Файл", "Изменить", "Вид", "Вставка", "Среда выполнения", "Инструменты", and "Справка". A "Подключиться" button is visible in the top right. The bottom status bar shows the system clock as 23:11 on 16.05.2025.

Выполнение лабораторной работы

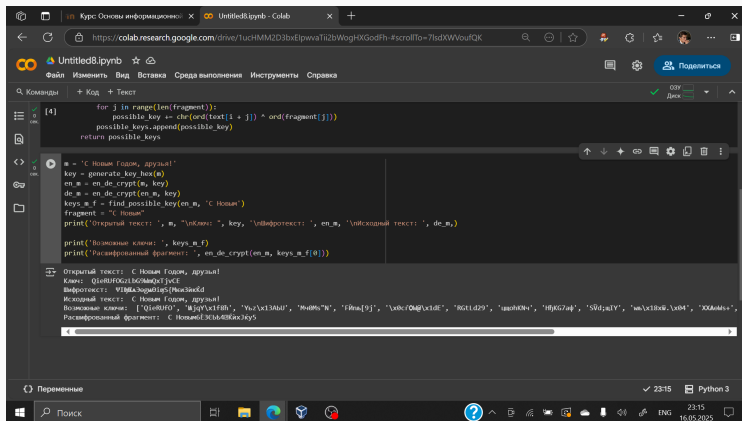
Нужно определить ключ, с помощью которого шифротекст может быть преобразован в некоторый фрагмент текста, представляющий собой один из возможных вариантов прочтения открытого текста. Для этого создаю функцию для нахождения возможных ключей для фрагмента текста.



```
[ ] def on_de_crypt(text, key):  
    new_text = ''  
    for i in range(len(text)):  
        new_text += chr(ord(text[i]) ^ ord(key[i % len(key)]))  
    return new_text  
  
def find_possible_key(text, fragment):  
    possible_keys = []  
    for i in range(len(text) - len(fragment) + 1):  
        possible_key = ''  
        for j in range(len(fragment)):  
            possible_key += chr(ord(text[i + j]) ^ ord(fragment[j]))  
        possible_keys.append(possible_key)  
    return possible_keys
```

Выполнение лабораторной работы

Проверка работы всех функций. Шифрование и дешифрование происходит верно, как и нахождение ключей, с помощью которых можно расшифровать верно только кусок текста



```
[4]: for j in range(len(fragment)):
    possible_key += chr(ord(text[i + j]) ^ ord(fragment[j]))
    possible_keys.append(possible_key)
    return possible_keys

m = 'С Новым Годом, друзья!'
key = generate_key_hex(m)
en_m = en_de_crypt(m, key)
de_m = en_de_crypt(en_m, key)
keys_m_f = find_possible_key(en_m, 'С Новым')
fragment = 'С Новым'
print('Открытый текст: ', m, '\nКлюч: ', key, '\nШифротекст: ', en_m, '\nИсходный текст: ', de_m,)

print('Возможные ключи: ', keys_m_f)
print('Расшифрованный фрагмент: ', en_de_crypt(en_m, keys_m_f[0]))

Открытый текст: С Новым Годом, друзья!
Ключ: Q1eWUFGz1bG9MmQx1jvCE
Шифротекст: V1B1kA3oym1a5(7m3kxkd
Исходный текст: С Новым Годом, друзья!
Возможные ключи: ['Q1eWUFG', 'MjRqVx1fBh', 'Yx1x13ABU', '9mB9b'N', 'F1m[9]', 'xbrC0MPlx1dE', 'RGLd29', 'm0hK4', 'tBjKG7aP', 'SvD;uIV', 'm\1x1BxW.\1aB4', 'X0MmM+']
Расшифрованный фрагмент: С Новым6L3x1b43kx3ky5
```

В ходе выполнения данной лабораторной работы мной было освоено на практике применение режима однократного гаммирования.